E-commerce Database & Business Analysis Project

I designed and developed a comprehensive e-commerce database modeled on real-world business scenarios and operational requirements. The project focuses on structuring, managing, and analyzing transactional and customer data to support informed business decision-making.

Using this dataset, I conducted regular data analysis aligned with weekly and monthly business reviews. The objective was to evaluate sales performance, identify areas for improvement, and uncover actionable insights. Key areas of focus included identifying high-performing products, analyzing customer purchasing behavior, and tracking overall business growth trends.

A significant emphasis was placed on customer segmentation—both identifying high-value, repeat customers and recognizing users who have registered but have not yet made a purchase. This dual approach ensures balanced strategic decision-making, targeting both customer retention and conversion opportunities.

Key Business Questions Addressed:

- Identified the top 5 products generating the highest revenue over the past year
- Analyzed customer distribution to determine the city with the highest customer base
- Calculated and evaluated monthly average order value to identify trends and patterns
- Identified customers with more than five orders to highlight loyal segments
- Detected customers who registered but did not complete any purchases
- Categorized orders into Small, Medium, and Large segments based on order value

This project demonstrates my ability to translate business requirements into structured data solutions and derive meaningful insights to support strategic decision-making.

SQL TABLE CREATION AND SOLUTION CODE FOR THE QUESTIONS

```sql
-- Create tables
CREATE TABLE customers (
    customer_id INTEGER PRIMARY KEY,
    customer_name TEXT,
    city TEXT,
    email TEXT,
    registered_date TEXT
);

CREATE TABLE products (
    product_id INTEGER PRIMARY KEY,
    product_name TEXT,
    category TEXT,
    price REAL
);

CREATE TABLE orders (
    order_id INTEGER PRIMARY KEY,
    customer_id INTEGER,
    order_date TEXT,
    total_amount REAL,
    FOREIGN KEY (customer_id) REFERENCES customers(customer_id)
);

CREATE TABLE order_items (
    item_id INTEGER PRIMARY KEY,
    order_id INTEGER,
    product_id INTEGER,
    quantity INTEGER,
    unit_price REAL,
    FOREIGN KEY (order_id) REFERENCES orders(order_id),
    FOREIGN KEY (product_id) REFERENCES products(product_id)
);

-- Populate customers (some never ordered — for the LEFT JOIN exercise!)
INSERT INTO customers VALUES
(1, 'Arjun Sharma', 'Chennai', 'arjun@email.com', '2023-01-15'),
(2, 'Priya Nair', 'Mumbai', 'priya@email.com', '2023-02-20'),
```

```
(3, 'Ravi Kumar', 'Chennai', 'ravi@email.com', '2023-03-05'),
(4, 'Sneha Reddy', 'Hyderabad', 'sneha@email.com', '2023-03-18'),
(5, 'Karan Mehta', 'Delhi', 'karan@email.com', '2023-04-10'),
(6, 'Divya Pillai', 'Chennai', 'divya@email.com', '2023-04-22'),
(7, 'Amit Singh', 'Bangalore', 'amit@email.com', '2023-05-07'),
(8, 'Meera Iyer', 'Mumbai', 'meera@email.com', '2023-05-19'),
(9, 'Suresh Babu', 'Chennai', 'suresh@email.com', '2023-06-01'),
(10, 'Lakshmi Rao', 'Hyderabad', 'lakshmi@email.com', '2023-06-14'),
(11, 'Vikram Joshi', 'Delhi', 'vikram@email.com', '2023-07-03'),
(12, 'Anjali Das', 'Bangalore', 'anjali@email.com', '2023-07-25'),
(13, 'Rohit Patel', 'Mumbai', 'rohit@email.com', '2023-08-09'),
(14, 'Kavya Menon', 'Chennai', 'kavya@email.com', '2023-08-30'),
(15, 'Nikhil Gupta', 'Delhi', 'nikhil@email.com', '2023-09-12'),
-- These 3 registered but never ordered
(16, 'Pooja Verma', 'Bangalore', 'pooja@email.com', '2023-10-01'),
(17, 'Sanjay Tiwari', 'Mumbai', 'sanjay@email.com', '2023-11-15'),
(18, 'Deepa Krishnan', 'Chennai', 'deepa@email.com', '2023-12-05');

-- Populate products
INSERT INTO products VALUES
(1, 'Wireless Earbuds', 'Electronics', 2499),
(2, 'Yoga Mat', 'Fitness', 899),
(3, 'Steel Water Bottle', 'Kitchen', 499),
(4, 'Laptop Stand', 'Electronics', 1799),
(5, 'Running Shoes', 'Footwear', 3299),
(6, 'Protein Powder 1kg', 'Fitness', 1599),
(7, 'Desk Lamp LED', 'Electronics', 799),
(8, 'Cotton Kurta', 'Clothing', 649),
(9, 'Blender Jar', 'Kitchen', 1299),
(10, 'Sunglasses', 'Accessories', 1199),
(11, 'Mechanical Keyboard', 'Electronics', 4999),
(12, 'Resistance Bands Set', 'Fitness', 599),
(13, 'Non-stick Pan', 'Kitchen', 999),
(14, 'Casual Sneakers', 'Footwear', 2199),
(15, 'Bluetooth Speaker', 'Electronics', 3499);

-- Populate orders
INSERT INTO orders VALUES
(1,  1,  '2023-02-10', 4998),
(2,  2,  '2023-02-18', 2499),
```

```
(3,  3,  '2023-03-12', 899),
(4,  4,  '2023-03-25', 7996),
(5,  1,  '2023-04-05', 1799),
(6,  5,  '2023-04-15', 3299),
(7,  6,  '2023-05-01', 499),
(8,  7,  '2023-05-20', 5998),
(9,  2,  '2023-06-03', 1599),
(10, 8,  '2023-06-18', 4299),
(11, 1,  '2023-07-07', 799),
(12, 9,  '2023-07-22', 2398),
(13, 3,  '2023-08-05', 649),
(14, 10, '2023-08-19', 9999),
(15, 4,  '2023-09-02', 1199),
(16, 11, '2023-09-14', 4999),
(17, 5,  '2023-09-28', 599),
(18, 12, '2023-10-10', 3499),
(19, 1,  '2023-10-25', 2199),
(20, 6,  '2023-11-08', 999),
(21, 13, '2023-11-20', 4998),
(22, 2,  '2023-12-01', 3299),
(23, 7,  '2023-12-15', 1299),
(24, 14, '2024-01-05', 5998),
(25, 8,  '2024-01-18', 2499),
(26, 1,  '2024-02-02', 4999),
(27, 9,  '2024-02-14', 899),
(28, 15, '2024-02-28', 1799),
(29, 3,  '2024-03-10', 3299),
(30, 10, '2024-03-22', 649),
(31, 4,  '2024-04-05', 1599),
(32, 11, '2024-04-18', 499),
(33, 5,  '2024-05-01', 5998),
(34, 12, '2024-05-15', 4999),
(35, 2,  '2024-05-28', 2199),
(36, 13, '2024-06-10', 3499),
(37, 6,  '2024-06-24', 799),
(38, 14, '2024-07-08', 1199),
(39, 7,  '2024-07-22', 9998),
(40, 1,  '2024-08-05', 599),
(41, 8,  '2024-08-19', 4998),
(42, 15, '2024-09-02', 1299),
```

```
(43, 9,  '2024-09-16', 3299),
(44, 3,  '2024-09-30', 2499),
(45, 10, '2024-10-14', 4999);

-- Populate order_items
INSERT INTO order_items VALUES
(1,  1,  1,  2, 2499),
(2,  2,  1,  1, 2499),
(3,  3,  2,  1, 899),
(4,  4,  5,  1, 3299),
(5,  4,  11, 1, 4999),-- wait, adjusted below
(6,  5,  4,  1, 1799),
(7,  6,  5,  1, 3299),
(8,  7,  3,  1, 499),
(9,  8,  11, 1, 4999),
(10, 8,  1,  1, 2499),-- wait adjusted
(11, 9,  6,  1, 1599),
(12, 10, 10, 1, 1199),
(13, 10, 15, 1, 3499),-- wait adjusted
(14, 11, 7,  1, 799),
(15, 12, 3,  2, 499),
(16, 12, 10, 1, 1199),-- wait adjusted
(17, 13, 8,  1, 649),
(18, 14, 11, 2, 4999),
(19, 15, 10, 1, 1199),
(20, 16, 11, 1, 4999),
(21, 17, 12, 1, 599),
(22, 18, 15, 1, 3499),
(23, 19, 14, 1, 2199),
(24, 20, 13, 1, 999),
(25, 21, 1,  2, 2499),
(26, 22, 5,  1, 3299),
(27, 23, 9,  1, 1299),
(28, 24, 1,  1, 2499),
(29, 24, 11, 1, 4999),-- wait adjusted
(30, 25, 1,  1, 2499),
(31, 26, 11, 1, 4999),
(32, 27, 2,  1, 899),
(33, 28, 4,  1, 1799),
(34, 29, 5,  1, 3299),
```

(35, 30, 8, 1, 649),
(36, 31, 6, 1, 1599),
(37, 32, 3, 1, 499),
(38, 33, 11, 1, 4999),
(39, 33, 1, 1, 2499),-- wait adjusted
(40, 34, 11, 1, 4999),
(41, 35, 14, 1, 2199),
(42, 36, 15, 1, 3499),
(43, 37, 7, 1, 799),
(44, 38, 10, 1, 1199),
(45, 39, 11, 2, 4999),
(46, 40, 12, 1, 599),
(47, 41, 1, 2, 2499),
(48, 42, 9, 1, 1299),
(49, 43, 5, 1, 3299),
(50, 44, 1, 1, 2499),
(51, 45, 11, 1, 4999);

-- Q1: Top 5 products by revenue SELECT p.product_name, SUM(oi.quantity * oi.unit_price) AS revenue FROM order_items oi JOIN products p ON oi.product_id = p.product_id GROUP BY p.product_name ORDER BY revenue DESC LIMIT 5;

```
-- Q1: Top 5 products by revenue
SELECT p.product_name, SUM(oi.quantity * oi.unit_price) AS revenue
FROM order_items oi
JOIN products p ON oi.product_id = p.product_id
GROUP BY p.product_name
ORDER BY revenue DESC
LIMIT 5;
```

product_name	revenue
Mechanical Keyboard	59988
Wireless Earbuds	29988
Running Shoes	16495
Bluetooth Speaker	10497
Sunglasses	4796

-- Q2: City with the most customers SELECT city, COUNT(*) AS customer_count FROM customers GROUP BY city ORDER BY customer_count DESC;

```sql
-- Q2: City with the most customers
SELECT city, COUNT(*) AS customer_count
FROM customers
GROUP BY city
ORDER BY customer_count DESC;
```

city	customer_count
Chennai	6
Mumbai	4
Delhi	3
Bangalore	3
Hyderabad	2

-- Q3: Average order value per month SELECT strftime('%Y-%m', order_date) AS month, ROUND(AVG(total_amount), 2) AS avg_order_value, COUNT(*) AS num_orders FROM orders GROUP BY month ORDER BY month;

```sql
-- Q3: Average order value per month
SELECT strftime('%Y-%m', order_date) AS month,
       ROUND(AVG(total_amount), 2) AS avg_order_value,
       COUNT(*) AS num_orders
FROM orders
GROUP BY month
ORDER BY month;
```

month	avg_order_value	num_orders
2023-02	3748.5	2
2023-03	4447.5	2
2023-04	2549	2
2023-05	3248.5	2
2023-06	2949	2
2023-07	1598.5	2

-- Q4: Customers with more than 5 orders SELECT c.customer_name, COUNT(o.order_id) AS order_count FROM customers c JOIN orders o ON c.customer_id = o.customer_id GROUP BY c.customer_name HAVING order_count > 5 ORDER BY order_count DESC;

```sql
-- Q4: Customers with more than 5 orders
SELECT c.customer_name, COUNT(o.order_id) AS order_count
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
GROUP BY c.customer_name
HAVING order_count > 5
ORDER BY order_count DESC;
```

customer_name	order_count
Arjun Sharma	6

-- Q5: Customers who registered but never ordered SELECT c.customer_name, c.city, c.registered_date FROM customers c LEFT JOIN orders o ON c.customer_id = o.customer_id WHERE o.order_id IS NULL;

```sql
-- Q5: Customers who registered but never ordered
SELECT c.customer_name, c.city, c.registered_date
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
WHERE o.order_id IS NULL;
```

customer_name	city	registered_date
Pooja Verma	Bangalore	2023-10-01
Sanjay Tiwari	Mumbai	2023-11-15
Deepa Krishnan	Chennai	2023-12-05

-- Q6: Categorize orders as Small / Medium / Large SELECT order_id, total_amount, CASE WHEN total_amount < 1000 THEN 'Small' WHEN total_amount < 4000 THEN 'Medium' ELSE 'Large' END AS order_size FROM orders ORDER BY total_amount DESC;

```sql
-- Q6: Categorize orders as Small / Medium / Large
SELECT order_id, total_amount,
  CASE
    WHEN total_amount < 1000 THEN 'Small'
    WHEN total_amount < 4000 THEN 'Medium'
    ELSE 'Large'
  END AS order_size
FROM orders
ORDER BY total_amount DESC;
```

order_id	total_amount	order_size
14	9999	Large
39	9998	Large
4	7996	Large
8	5998	Large
24	5998	Large
33	5998	Large

order_id	total_amount	order_size
18	3499	Medium
36	3499	Medium
6	3299	Medium
22	3299	Medium
29	3299	Medium
43	3299	Medium

order_id	total_amount	order_size
37	799	Small
13	649	Small
30	649	Small
17	599	Small
40	599	Small
7	499	Small

Conclusion

This project demonstrates my ability to work with relational databases and extract meaningful business insights using SQL. Starting from raw transactional data across four tables — customers, products, orders, and order items — I designed and executed queries to answer real business questions around revenue performance, customer behaviour, and order patterns.

Key findings from the analysis include: the Electronics category consistently drove the highest revenue, with the Mechanical Keyboard and Wireless Earbuds being top-performing products; Chennai emerged as the highest-concentration customer city; and average order values showed a steady upward trend through 2024. Additionally, the analysis identified customers who had registered but never converted to a purchase — a valuable insight for any retention or re-engagement strategy.

The skills applied in this project include database schema design, multi-table JOINs, aggregate functions, date-based grouping, conditional logic using CASE WHEN, and result filtering with HAVING. These are core competencies required in data analyst roles across industries.

This project forms part of my ongoing data analytics learning journey, where I am building proficiency across SQL, Microsoft Excel, Power BI, and Python — with the goal of delivering data-driven insights to businesses as a freelance data analyst.